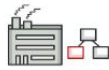




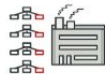
Patrones de Diseño

Creational patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.



Factory Method



Abstract Factory



Builder



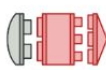
Prototype



Singleton

Structural patterns

These patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.



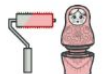
Adapter



Bridge



Composite



Decorator



Facade



Flyweight



Proxy

Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.



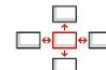
Chain of Responsibility



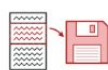
Command



Iterator



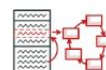
Mediator



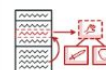
Memento



Observer



State



Strategy



Template Method



Visitor

Patrón Creacional: Factory

El patrón Factory (Fábrica) es uno de los más utilizados en C#. Su propósito principal es desacoplar la creación de un objeto de su implementación.

En lugar de usar el operador `new` directamente en tu lógica de negocio (lo cual te ata a una clase específica), delegas esa responsabilidad a una "fábrica".

El Escenario: Sistema de Notificaciones

Imagina que tu aplicación necesita enviar notificaciones. Al principio solo envías Email, pero pronto te piden SMS y luego WhatsApp.

1. La Interfaz (El Producto)

Primero definimos una interfaz común para todos los tipos de notificaciones.

```
csharp
public interface INotification
{
    void Send(string message);
}
```

2. Implementaciones Concretas (Productos Concretos)

Creemos las clases que realmente hacen el trabajo.

```
public class EmailNotification : INotification
{
    public void Send(string message) => Console.WriteLine($"Enviando Email: {message}");
}
```

```
public class SmsNotification : INotification
{
    public void Send(string message) => Console.WriteLine($"Enviando SMS: {message}");
}
```

3. La Fábrica (Simple Factory)

Aquí es donde ocurre la "magia". La fábrica decide qué objeto instanciar.

```
public static class NotificationFactory
{
    // Usamos el switch moderno de C# (Pattern Matching)

    public static INotification CreateNotification(string type)
    {
        return type.ToLower() switch
        {
            "email" => new EmailNotification(),
            "sms"   => new SmsNotification(),
            _       => throw new ArgumentException("Tipo de notificación no soportado")
        };
    }
}
```

4. Uso en el Código (Cliente)

Como Senior, fíjate en esto: El código que usa la notificación no sabe (ni le importa) si es un Email o un SMS.

```
public class NotificationService
{
    public void Execute(string type, string message)
    {
        // El cliente NO hace: new EmailNotification()

        // El cliente le pide a la fábrica el objeto

        INotification notification = NotificationFactory.CreateNotification(type);

        notification.Send(message);
    }
}
```

Diferencia importante: Factory Method vs Abstract Factory

Lo que te mostré arriba es un Simple Factory. En C# avanzado verás:

- Factory Method: Se basa en herencia. La clase base define el método, pero las subclases deciden qué instancia crear.
- Abstract Factory: Es una "fábrica de fábricas". Se usa cuando necesitas crear familias de objetos relacionados (ej: un set de botones y ventanas que sean todos estilo "Windows" o todos estilo "Mac").

Dato extra para C#: En .NET moderno, muchas veces el Contenedor de Inyección de Dependencias (DI) actúa como una fábrica gigante, pero entender el patrón manual es vital para escenarios donde el objeto depende de datos que solo conoces en tiempo de ejecución.

Patrón Creacional: Builder

El patrón Builder (Constructor) es fundamental cuando te enfrentas a la creación de objetos complejos que tienen muchas partes opcionales o configuraciones distintas.

Quizás has visto el "Antipatrón del Constructor Telescópico": una clase con 5 constructores diferentes porque a veces necesitas el parámetro A, otras el B, y otras el A+C. El Builder elimina este caos.

El Escenario: Configuración de un Servidor

Imagina que estás construyendo un objeto `Server`. Un servidor puede tener CPU, RAM, Almacenamiento, ¿es GPU?, ¿tiene SSL?, ¿tiene Backup?.

1. El Producto (El objeto complejo)

Queremos que este objeto sea, idealmente, inmutable una vez creado.

```
public class Server
{
    public string CPU { get; internal set; }

    public int RAM { get; internal set; }

    public int Storage { get; internal set; }

    public bool IsGpuEnabled { get; internal set; }

    public bool HasSsl { get; internal set; }

    public override string ToString() =>
        $"Server: {CPU}, {RAM}GB RAM, {Storage}GB SSD, GPU: {IsGpuEnabled}, SSL: {HasSsl}";
}
```

2. El Builder (La interfaz fluida)

Aquí usamos una técnica llamada Fluent Interface (encadenamiento de métodos), que es el estándar moderno en C#.

```
public interface IServerBuilder
{
    IServerBuilder SetCPU(string cpu);
    IServerBuilder SetRAM(int ram);
    IServerBuilder SetStorage(int storage);
    IServerBuilder EnableGpu();
    IServerBuilder WithSsl();
    Server Build(); // El método que finalmente entrega el objeto
}

public class ServerBuilder : IServerBuilder
{
    private Server _server = new Server();

    public IServerBuilder SetCPU(string cpu) {
        _server.CPU = cpu;
        return this; // Retornamos la misma instancia para encadenar
    }

    public IServerBuilder SetRAM(int ram) {
        _server.RAM = ram;
        return this;
    }

    public IServerBuilder SetStorage(int storage) {
        _server.Storage = storage;
        return this;
    }
}
```

```
public IServerBuilder EnableGpu() {
    _server.IsGpuEnabled = true;
    return this;
}

public IServerBuilder WithSsl() {
    _server.HasSsl = true;
    return this;
}

public Server Build() {
    // Aquí podrías validar que el objeto sea consistente antes de entregarlo
    if (string.IsNullOrEmpty(_server.CPU)) throw new Exception("CPU es obligatoria");

    return _server;
}
```

3. Uso en el Código (Legibilidad total)

Mira qué limpio queda el código del cliente comparado con un constructor lleno de `null, null, true, false, 512`.

```
var builder = new ServerBuilder();

// Podemos crear un servidor básico
Server webServer = builder
    .SetCPU("Intel Xeon")
    .SetRAM(16)
    .SetStorage(512)
    .WithSsl()
    .Build();

// O un servidor potente de IA
Server aiServer = new ServerBuilder()
    .SetCPU("AMD EPYC")
    .SetRAM(128)
    .SetStorage(2048)
    .EnableGpu()
    .Build();

Console.WriteLine(webServer);
Console.WriteLine(aiServer);
```

Patrón Creacional: Prototype

El patrón Prototype (Prototipo) es un patrón creacional que se usa cuando el costo de crear un objeto nuevo desde cero es muy caro (en términos de tiempo o recursos) o cuando quieres evitar la complejidad de un constructor lleno de parámetros.

En lugar de hacer un `new`, clonas un objeto existente que ya está configurado.

El Escenario: Configuración de Personajes en un Videojuego

Imagina que tienes un objeto `Stormtrooper`. Crear uno implica cargar texturas pesadas, sonidos y configuraciones de IA desde el disco. Si necesitas un ejército de 100, no quieres hacer `new` 100 veces y recargar todo. Quieres copiar uno que ya esté en memoria y quizás cambiarle solo el arma.

1. La Interfaz (IPrototype)

En C#, históricamente existía la interfaz `ICloneable`, pero hoy en día se recomienda crear la propia o usar métodos específicos para tener control total sobre la Copia Superficial (Shallow Copy) vs Copia Profunda (Deep Copy).

```
public abstract class EnemyPrototype
{
    public string Name { get; set; }

    public Weapon Weapon { get; set; } // Clase de referencia

    // El método clave

    public abstract EnemyPrototype Clone(bool deepCopy);
}

public class Weapon
{
    public string Description { get; set; }
}
```

2. Implementación Concreta

Aquí es donde manejamos la lógica de clonación.

```
public class Stormtrooper : EnemyPrototype
{
    public Stormtrooper(string name, string weaponName)
    {
        this.Name = name;
        this.Weapon = new Weapon { Description = weaponName };
        // Imagina aquí un proceso pesado de carga de recursos...
        Console.WriteLine($"-> Proceso pesado: Cargando recursos para {Name}...");
    }

    public override EnemyPrototype Clone(bool deepCopy)
    {
        if (deepCopy)
        {
            // COPIA PROFUNDA: Creamos nuevas instancias de los objetos internos
            var clone = (Stormtrooper)this.MemberwiseClone();
            clone.Weapon = new Weapon { Description = this.Weapon.Description };
            return clone;
        }
        else
        {
            // COPIA SUPERFICIAL: Copia el objeto pero comparte las referencias internas
            return (EnemyPrototype)this.MemberwiseClone();
        }
    }
}
```

Aclaración:

`MemberwiseClone()` es un método protegido (protected) de la clase `System.Object`. Esto significa que todas las clases en C# lo heredan, pero solo puedes usarlo dentro de la propia clase.

Su función es crear una Copia Superficial (Shallow Copy) del objeto actual.

¿Cómo funciona internamente?

Cuando llamas a `MemberwiseClone()`, .NET crea un nuevo objeto de la misma clase y copia los campos uno a uno:

1. Tipos de Valor (`int`, `bool`, `struct`, `double`, etc.): Se realiza una copia bit a bit del dato. El nuevo objeto tiene su propia copia independiente.
2. Tipos de Referencia (`class`, `List`, `Array`, etc.): Se copia la dirección de memoria (el puntero). Aquí es donde está el peligro: tanto el objeto original como el clon apuntan al mismo objeto en el heap.

3. Uso en el Código

Aquí es donde ves la utilidad: ahorras ciclos de CPU y mantienes el estado base.

```
// 1. Creamos el prototipo original (Costo alto de CPU/Memoria)
var originalTrooper = new Stormtrooper("Soldado Base", "Rifle Laser");

// 2. Necesitamos un batallón. Clonamos en lugar de crear de cero.
// Esto es instantáneo comparado con el 'new' original.
var soldier1 = (Stormtrooper)originalTrooper.Clone(deepCopy: true);
soldier1.Name = "Soldado Alfa";

var soldier2 = (Stormtrooper)originalTrooper.Clone(deepCopy: true);
soldier2.Name = "Soldado Beta";
soldier2.Weapon.Description = "Lanzallamas";

Console.WriteLine($"Original: {originalTrooper.Name} con {originalTrooper.Weapon.Description}");
Console.WriteLine($"Clon 1: {soldier1.Name} con {soldier1.Weapon.Description}");
Console.WriteLine($"Clon 2: {soldier2.Name} con {soldier2.Weapon.Description}");
```

Patrón Creacional: Singleton

Con el Singleton, cerramos el grupo de los 5 patrones creacionales (Factory Method, Abstract Factory, Builder, Prototype y Singleton).

El Singleton es probablemente el más famoso, pero también el más criticado. Un Senior debe saber no solo cómo implementarlo, sino cuándo evitarlo.

El Escenario: Gestor de Logs (Logger)

No quieres tener 10 archivos de log abiertos por 10 instancias diferentes. Quieres una única instancia en toda la aplicación que centralice la escritura de mensajes.

1. Implementación "Clásica" (Thread-Safe)

En aplicaciones multihilo (como ASP.NET), un Singleton mal implementado puede crear dos instancias si dos hilos entran al mismo tiempo. Esta es la forma robusta usando un `lock`.

```
public sealed class Logger
{
    // 1. Constructor privado: nadie puede hacer 'new Logger()'
    private Logger() { }

    private static Logger _instance;
    private static readonly object _lock = new object();

    // 2. Punto de acceso global
    public static Logger Instance
    {
        get
        {
            // Double-check locking para performance
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                    {
                        _instance = new Logger();
                    }
                }
            }
            return _instance;
        }
    }

    public void Log(string message) => Console.WriteLine($"Log: {message}");
}
```

2. La forma moderna en C# (`Lazy<T>`)

Desde .NET 4.0, existe una forma mucho más elegante y segura de hacerlo. `Lazy<T>` garantiza que el objeto solo se cree cuando se necesite y es thread-safe por defecto.

```
public sealed class ModernLogger
{
    private static readonly Lazy<ModernLogger> _lazy =
        new Lazy<ModernLogger>(() => new ModernLogger());

    public static ModernLogger Instance => _lazy.Value;

    private ModernLogger() { }

    public void Log(string message) => Console.WriteLine($"Modern Log: {message}");
}
```

3. Otra forma moderna en .Net CORE (DI)

Registro como Singleton en Inyección de Dependencias (DI).

En .NET moderno (ASP.NET Core), no creas la clase como Singleton internamente. Creas una clase normal y la registras en el contenedor de servicios:

```
csharp
```

```
services.AddSingleton<ILogger, MyLogger>();
```

Patrón Estructural: Adapter

El Concepto

El Adapter actúa como un traductor. Permite que dos interfaces que son incompatibles puedan trabajar juntas. Es exactamente igual que un adaptador de corriente cuando viajas: el enchufe de la pared tiene una forma, tu cargador tiene otra, y el adaptador hace el puente.

El Escenario: Sistema de Procesamiento de Pagos

Imagina que tu aplicación procesa pagos usando una interfaz estándar `IPaymentProcessor`. Todo tu sistema está construido sobre esa interfaz.

Un día, el negocio decide integrar un nuevo proveedor llamado "MegaPay". Sin embargo, la librería de MegaPay viene de un tercero y sus métodos no se llaman igual que los tuyos. No puedes cambiar el código de MegaPay (porque es una DLL externa) y no quieres cambiar todo tu código base para adaptarte a ellos.

1. Tu Interfaz Actual (Target)

Esto es lo que tu aplicación ya sabe usar.

```
public interface IPaymentProcessor  
{  
  
    // Tu sistema espera este método  
  
    void ProcessPayment (decimal amount);  
  
}
```

2. La Clase Externa (Adapter)

Esta es la librería de "MegaPay" que tiene nombres de métodos distintos y quizás lógica diferente.

```
public class MegaPayApi
{
    public void MakeTransaction (double total)
    {
        Console.WriteLine($"Pago procesado con MegaPay API: $ {total}");
    }
}
```


3. El Adaptador (Adapter)

Aquí es donde aplicamos el patrón. Creamos una clase que implementa tu interfaz, pero por dentro llama a la librería externa.

```
public class MegaPayAdapter : IPaymentProcessor
{
    private readonly MegaPayApi _megaPayApi;

    public MegaPayAdapter(MegaPayApi api)
    {
        _megaPayApi = api;
    }

    public void ProcessPayment(decimal amount)
    {
        // 1. Aquí el adaptador hace la "traducción" de tipos (decimal a double)
        double amountAsDouble = (double)amount;

        // 2. Aquí traduce el nombre del método
        _megaPayApi.MakeTransaction(amountAsDouble);
    }
}
```

4. Uso en el Código (Cliente)

Gracias al adaptador, tu código principal ni se entera de que está usando una API externa con métodos raros.

```
public class CheckoutService
{
    public void FinalizeOrder(IPaymentProcessor processor, decimal total)
    {
        // El servicio de checkout solo conoce "ProcessPayment"
        processor.ProcessPayment(total);
    }
}

// --- En el Main o Composition Root ---
var externalApi = new MegaPayApi();
IPaymentProcessor adapter = new MegaPayAdapter(externalApi);

var checkout = new CheckoutService();
checkout.FinalizeOrder(adapter, 100.50m);
```

Diferencia clave: Adapter vs Facade (Fachada)

Es común confundirlos, pero hay una diferencia de intención:

- Adapter: Se usa para que algo encaje en una interfaz específica que ya tienes. (Ej: "Haz que MegaPay parezca un `IPaymentProcessor`").
- Facade: Se usa para crear una interfaz simplificada encima de un sistema complejo de muchas clases. (Ej: "Crea un método `UnicoBotonComprar()` que por dentro llame a Inventario, Envío, Pago y Email").

Diferencia clave: Facade vs Adapter

Es la pregunta típica en entrevistas Senior:

- Adapter: Cambia la interfaz de un objeto para que parezca otro (Traducción).
- Facade: Crea una interfaz nueva y simple para manejar muchos objetos a la vez (Simplificación).

Patrón Estructural: Facade

El patrón Facade (Fachada) es uno de los patrones estructurales más agradecidos y utilizados. Como su nombre indica, consiste en poner una "cara bonita" o simple delante de un sistema que por dentro es horrible o muy complejo.

El Concepto

En lugar de obligar al cliente a interactuar con 10 clases diferentes, crear instancias de todas, configurar sus estados y llamar a sus métodos en el orden correcto, creas una sola clase (la Fachada) que ofrece un método simple que hace todo eso por debajo.

El Escenario: Sistema de Pedidos (E-commerce)

Para procesar un pedido, tu sistema necesita:

1. Inventory: Verificar si hay stock.
2. Payment: Procesar el cobro.
3. Shipping: Generar la etiqueta de envío.
4. Notification: Enviar el email de confirmación.

Sin una fachada, el programador que quiere hacer una venta tiene que conocer todas estas clases.

1. El Subsistema Complejo (Las piezas internas)

```
public class Inventory {  
  
    public bool CheckStock(string productId) => true;  
  
}  
  
public class PaymentProcessor {  
  
    public bool Charge(decimal amount) => true;  
  
}  
  
public class ShippingService {  
  
    public void CreateLabel() => Console.WriteLine("Etiqueta de envío creada.");  
  
}  
  
public class EmailService {  
  
    public void SendConfirmation() => Console.WriteLine("Email enviado.");  
  
}
```

2. La Fachada (Facade)

Esta clase orquesta todas las anteriores. El cliente solo verá esta clase.

```
public class OrderFacade
{
    private readonly Inventory _inventory = new();
    private readonly PaymentProcessor _payment = new();
    private readonly ShippingService _shipping = new();
    private readonly EmailService _email = new();

    // El método "mágico" que simplifica todo
    public bool PlaceOrder(string productId, decimal price)
    {
        Console.WriteLine("Iniciando proceso de pedido...");

        if (!_inventory.CheckStock(productId)) return false;
        if (!_payment.Charge(price)) return false;

        _shipping.CreateLabel();
        _email.SendConfirmation();

        Console.WriteLine("¡Pedido completado con éxito!");
        return true;
    }
}
```

3. Uso en el Código (Cliente)

Fíjate en la diferencia. El cliente no sabe nada de inventarios ni de logística.

```
// Sin Facade: Tendrías que instanciar 4 clases y llamar a 4-5 métodos.
```

```
// Con Facade:
```

```
var store = new OrderFacade ();
```

```
store.PlaceOrder ("Laptop-123", 1200.00m);
```

Patrón Estructural: Proxy

El patrón Proxy (Sustituto) es un patrón estructural que actúa como un intermediario o "pasamanos". Proporciona un objeto que controla el acceso a otro objeto (el objeto real).

El Concepto

El Proxy tiene la misma interfaz que el objeto real. El cliente no sabe que está hablando con un intermediario. El Proxy recibe la petición y decide si:

1. La pasa al objeto real.
2. La bloquea (Seguridad).
3. Devuelve un resultado guardado (Cache).
4. Retrasa la creación del objeto real hasta que sea estrictamente necesario (Lazy Loading).

El Escenario: Acceso a una Base de Datos "Pesada"

Imagina que tienes una clase que descarga reportes pesados de una base de datos. No quieres que cualquier usuario pueda ejecutarlos, y tampoco quieres descargar el mismo reporte 10 veces en un minuto.

1. La Interfaz Común

Tanto el objeto real como el proxy deben lucir iguales.

```
public interface IReportService  
{  
  
    void DisplayReport (string userId);  
  
}
```

2. El Objeto Real (Heavy Object)

Este es el objeto que queremos proteger o cuyo acceso queremos optimizar.

```
public class RealReportService : IReportService
{
    public RealReportService ()
    {
        // Simula una carga muy pesada al instanciar

        Console.WriteLine ("-> Cargando datos masivos de la BD... (Muy lento)" );

        Thread.Sleep (2000);
    }

    public void DisplayReport (string userId)
    {
        Console.WriteLine ($"Mostrando el reporte detallado para el usuario {userId}.");
    }
}
```

3. El Proxy (El Intermediario)

Aquí añadimos la lógica de Control de Acceso y Lazy Loading.

```
public class ReportServiceProxy : IReportService
{
    private RealReportService _realService; // El objeto real está oculto aquí
    private string _userRole;

    public ReportServiceProxy(string userRole)
    {
        _userRole = userRole;
    }

    public void DisplayReport(string userId)
    {
        // 1. Control de Seguridad (Protection Proxy)
        if (_userRole != "Admin")
        {
            Console.WriteLine("ERROR: Acceso denegado. Solo administradores pueden ver reportes.");
            return;
        }

        // 2. Carga Perezosa (Virtual Proxy)
        // Solo creamos el objeto pesado si el usuario realmente tiene permiso y lo pide
        if (_realService == null)
        {
            _realService = new RealReportService();
        }

        _realService.DisplayReport(userId);
    }
}
```

4. Uso en el Código

```
// Cliente 1: Intenta entrar como usuario normal
```

```
IReportService proxy1 = new ReportServiceProxy("User");
```

```
proxy1.DisplayReport("101"); // Resultado: Acceso denegado.
```

```
// Cliente 2: Entra como Admin
```

```
IReportService proxy2 = new ReportServiceProxy("Admin");
```

```
// En este momento el RealReportService aún NO se ha creado.
```

```
proxy2.DisplayReport("101"); // Aquí se crea y se muestra.
```

```
proxy2.DisplayReport("101"); // Aquí ya está creado, es instantáneo.
```

Diferencia clave: Proxy vs Decorator

A veces se confunden porque ambos "envuelven" a otro objeto:

- Proxy: Controla el acceso y gestiona el ciclo de vida del objeto (él decide cuándo crearlo o si dejarte pasar).
- Decorator: Se usa para añadir responsabilidades dinámicamente. El cliente suele crear el objeto real y luego se lo pasa al decorador para "tunearlo".

Patrón Estructural: Decorator

El patrón Decorator (Decorador) es uno de los favoritos de los arquitectos porque es la máxima expresión del principio de "Composición sobre Herencia".

El Concepto

Permite añadir funcionalidades a un objeto de forma dinámica sin alterar su código y sin crear una explosión de subclases. En lugar de heredar para extender, "envuelves" el objeto original en otro objeto que contiene la nueva lógica.

El Escenario: Sistema de Mensajería

Imagina que tienes un servicio que envía mensajes de texto. Luego te piden:

1. Que el mensaje se pueda Cifrar.
2. Que el mensaje se pueda Comprimir.
3. Que el mensaje se pueda Cifrar Y Comprimir.

Si usaras herencia, tendrías clases como `SmsEncrypted`, `SmsCompressed`, `SmsEncryptedAndCompressed` ... y si añades una función más, el número de clases se vuelve inmanejable.

1. La Interfaz Base (Component)

Define lo que todos los mensajes (simples o decorados) pueden hacer.

```
public interface IMessage  
{  
  
    string GetContent () ;  
  
}
```

2. El Objeto Real (Concrete Component)

La implementación básica que queremos "decorar".

```
public class TextMessage : IMessage
{
    private string _text;

    public TextMessage(string text) => _text = text;

    public string GetContent() => _text;
}
```


3. El Decorador Base (Decorator)

Esta clase es clave: implementa la interfaz Y contiene una instancia de la interfaz.

```
public abstract class MessageDecorator : IMessage
{
    protected IMessage _message; // El objeto envuelto

    public MessageDecorator(IMessage message)
    {
        _message = message;
    }

    public virtual string GetContent() => _message.GetContent();
}
```

4. Decoradores Concretos

Añaden la funcionalidad extra antes o después de llamar al objeto envuelto.

```
// Decorador 1: Cifrado
public class EncryptionDecorator : MessageDecorator
{
    public EncryptionDecorator(IMessage message) : base(message) { }

    public override string GetContent()
    {
        string content = base.GetContent();
        return $"[Cifrado: {Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(content))}]";
    }
}

// Decorador 2: Comprimido
public class CompressionDecorator : MessageDecorator
{
    public CompressionDecorator(IMessage message) : base(message) { }

    public override string GetContent()
    {
        return $"[Comprimido: {base.GetContent().Substring(0, 5)}...]";
    }
}
```

5. Uso en el Código (Como capas de cebolla)

Aquí es donde ocurre la magia. Puedes combinar los decoradores como quieras en tiempo de ejecución.

```
// 1. Mensaje simple
```

```
IMessage message = new TextMessage("Hola mundo");
```

```
// 2. Queremos cifrarlo
```

```
message = new EncryptionDecorator(message);
```

```
// 3. Y ahora también queremos comprimirlo (envolvemos al envoltorio)
```

```
message = new CompressionDecorator(message);
```

```
// El resultado final tiene todas las capas
```

```
Console.WriteLine(message.GetContent());
```

```
// Output: [Comprimido: [Cifrado: SG9sYSBtdW5kbw==]...]
```

Patrón Estructural: Bridge

El patrón Bridge (Puente) es quizás uno de los más difíciles de entender al principio, pero es una herramienta poderosa para evitar la explosión de clases en una jerarquía de herencia.

El Concepto

Su objetivo es desvincular una abstracción de su implementación, de modo que ambas puedan evolucionar de forma independiente.

En lugar de tener una jerarquía de clases gigante que intenta cubrir todas las combinaciones posibles, divides el problema en dos jerarquías separadas y las unes mediante un "puente" (una referencia).

El Escenario: Mandos a Distancia y Dispositivos

Imagina que tienes dos tipos de Controles Remotos (Básico y Avanzado) y dos tipos de Dispositivos (TV y Radio).

Si usaras herencia tradicional, necesitarías:

1. `TvRemoteBasic`
2. `TvRemoteAdvanced`
3. `RadioRemoteBasic`
4. `RadioRemoteAdvanced`

Si añades un dispositivo más (ej: Proyector), ¡necesitas 2 clases más! Esto crece exponencialmente ($M \times N$). El Bridge lo convierte en una suma ($M + N$).

1. La Implementación (La parte "sucia" o de hardware)

Definimos la interfaz para los dispositivos.

```
public interface IDevice
{
    void SetVolume(int percent);
    void PowerOn();
    void PowerOff();
}

public class Tv : IDevice
{
    public void PowerOn() => Console.WriteLine("TV: Encendida");
    public void PowerOff() => Console.WriteLine("TV: Apagada");
    public void SetVolume(int percent) => Console.WriteLine($"TV: Volumen al {percent}%");
}

public class Radio : IDevice
{
    public void PowerOn() => Console.WriteLine("Radio: Encendida");
    public void PowerOff() => Console.WriteLine("Radio: Apagada");
    public void SetVolume(int percent) => Console.WriteLine($"Radio: Volumen al {percent}%");
}
```

2. La Abstracción (La parte que usa el cliente)

El "Mando" tiene una referencia al "Dispositivo". Este es el Puente.

```
public class RemoteControl
{
    protected IDevice _device; // El puente

    public RemoteControl(IDevice device) => _device = device;

    public void TogglePower()
    {
        Console.WriteLine("Mando: Botón Power presionado");

        _device.PowerOn();
    }
}
```

3. Abstracción Refinada

Creamos una variante del mando sin tocar el código de los dispositivos.

```
public class AdvancedRemoteControl : RemoteControl
{
    public AdvancedRemoteControl(IDevice device) : base(device) { }

    public void Mute()
    {
        Console.WriteLine("Mando: Botón Mute presionado");
        _device.SetVolume(0);
    }
}
```

4. Uso en el Código

Mira cómo puedes mezclar y combinar cualquier mando con cualquier dispositivo en tiempo de ejecución.

```
// Queremos un mando básico para la TV
```

```
IDevice tv = new Tv();
```

```
var basicRemote = new RemoteControl(tv);
```

```
basicRemote.TogglePower();
```

```
// Queremos un mando avanzado para la Radio
```

```
IDevice radio = new Radio();
```

```
var advancedRemote = new AdvancedRemoteControl(radio);
```

```
advancedRemote.TogglePower();
```

```
advancedRemote.Mute();
```


Patrón Estructural: Composite

El patrón Composite (Compuesto) es el patrón por excelencia cuando necesitas trabajar con estructuras de árbol. Su objetivo es permitir que los clientes traten de la misma manera a un objeto individual que a una colección de objetos.

El Concepto

Imagina una estructura jerárquica donde tienes "hojas" (objetos simples) y "contenedores" (objetos que contienen hojas y otros contenedores). El patrón Composite dice: "Haz que ambos implementen la misma interfaz".

De esta forma, puedes ejecutar una operación en el nivel superior y esta se propagará recursivamente por toda la estructura sin que el cliente tenga que distinguir si está tocando un archivo o una carpeta.

El Escenario: Sistema de Archivos

Tienes Archivos y tienes Carpetas. Una carpeta puede contener archivos y otras carpetas. Queremos calcular el tamaño total de una carpeta o mostrar su contenido.

1. El Componente (La Interfaz Común)

Tanto el archivo como la carpeta deben lucir iguales para el cliente.

```
public abstract class FileSystemItem
{
    public string Name { get; protected set; }

    public FileSystemItem(string name) => Name = name;

    public abstract long GetSize(); // Operación común

    public abstract void Display(int indent); // Operación común
}
```

2. La Hoja (Leaf)

Es el objeto simple que no tiene hijos.

```
public class FileItem : FileSystemItem
{
    private long _size;

    public FileItem(string name, long size) : base(name)
    {
        _size = size;
    }

    public override long GetSize() => _size;

    public override void Display(int indent)
    {
        Console.WriteLine($"{new string('-', indent)} Archivo: {Name} ({_size} KB)");
    }
}
```

3. El Compuesto (Composite)

Es el objeto complejo que contiene una lista de `FileSystemItem`.

```
public class FolderItem : FileSystemItem
{
    private List<FileSystemItem> _children = new List<FileSystemItem>();

    public FolderItem(string name) : base(name) { }

    public void Add(FileSystemItem item) => _children.Add(item);
    public void Remove(FileSystemItem item) => _children.Remove(item);

    public override long GetSize()
    {
        // Recursividad: Suma el tamaño de todo lo que contiene
        return _children.Sum(child => child.GetSize());
    }

    public override void Display(int indent)
    {
        Console.WriteLine($"{new string('-', indent)} CARPETA: {Name}");
        foreach (var child in _children)
        {
            child.Display(indent + 2); // Propaga la llamada
        }
    }
}
```

4. Uso en el Código

Fíjate en que el cliente puede tratar a `root` (una carpeta) o a un archivo individual usando la misma firma de métodos.

```
// 1. Construimos la estructura
var root = new FolderItem("C:");
var appFolder = new FolderItem("Program Files");
var dataFolder = new FolderItem("Data");

var file1 = new FileItem("config.ini", 10);
var file2 = new FileItem("system.dll", 500);
var photo = new FileItem("vacaciones.jpg", 2000);

root.Add(appFolder);
root.Add(photo);
appFolder.Add(file1);
appFolder.Add(dataFolder);
dataFolder.Add(file2);

// 2. Usamos la estructura de forma uniforme
Console.WriteLine($"Tamaño total del disco: {root.GetSize()} KB");
root.Display(0);
```

Patrón Estructural: Flyweight

El patrón Flyweight (Peso Ligero) es el patrón estructural orientado específicamente a la optimización de memoria (RAM). Este patrón te resultará fascinante cuando trabajes en sistemas que manejan miles o millones de objetos similares.

El Concepto

El objetivo es evitar crear miles de objetos que contienen los mismos datos repetidos. En lugar de eso, extraemos los datos comunes y los compartimos entre muchos objetos.

Para que funcione, dividimos el estado del objeto en dos:

1. Estado Intrínseco: Los datos que son constantes y se pueden compartir (ej: el color de un icono, la textura de un árbol, el nombre de un departamento).
2. Estado Extrínseco: Los datos que cambian y son únicos para cada instancia (ej: las coordenadas X e Y, el nombre de un empleado específico).

El Escenario: Un Bosque en un Videojuego

Imagina que quieres renderizar 100,000 árboles. Cada árbol tiene:

- Un nombre ("Pino").
- Un color ("Verde oscuro").
- Una textura (un array de bytes de 10MB con la imagen de las hojas).
- Coordenadas (X, Y).

Si creas 100,000 objetos `Tree` y cada uno carga 10MB de textura, ¡necesitarías 1 Terabyte de RAM! El Flyweight soluciona esto.

1. El Flyweight (Estado compartido)

Esta clase contiene los datos pesados que se repiten. Debe ser **immutable**.

```
public class TreeType
{
    // Datos compartidos (pesados)
    public string Name { get; }
    public string Color { get; }
    public byte[] Texture { get; } // Imagina 10MB aquí

    public TreeType(string name, string color, byte[] texture)
    {
        Name = name;
        Color = color;
        Texture = texture;
    }

    public void Draw(int x, int y)
    {
        // Usa los datos compartidos (textura) para dibujar en una posición única
        Console.WriteLine($"Dibujando {Name} de color {Color} en ({x}, {y})");
    }
}
```

2. La Fábrica (Flyweight Factory)

Gestiona el "cache" de los flyweights. Si pides un tipo de árbol que ya existe, te devuelve el mismo.

```
public class TreeFactory
{
    private static Dictionary<string, TreeType> _treeTypes = new();

    public static TreeType GetTreeType(string name, string color, byte[] texture)
    {
        string key = $"{name}_{color}";

        if (!_treeTypes.ContainsKey(key))
        {
            Console.WriteLine($"=> Creando nuevo tipo de árbol: {name}");
            _treeTypes[key] = new TreeType(name, color, texture);
        }

        return _treeTypes[key];
    }
}
```


3. El Contexto (El objeto ligero)

Contiene solo la parte única (coordenadas) y una referencia al Flyweight.

```
public class Tree
{
    private int _x, _y;
    private TreeType _type; // Referencia al objeto compartido

    public Tree(int x, int y, TreeType type)
    {
        _x = x;
        _y = y;
        _type = type;
    }

    public void Draw() => _type.Draw(_x, _y);
}
```

4. Uso en el Código

```
var heavyTexture = new byte[10000000]; // 10MB simula la textura
var forest = new List<Tree>();

// Creamos 100,000 árboles
for (int i = 0; i < 100000; i++)
{
    // Solo se creará UN objeto TreeType en memoria, aunque haya 100,000 árboles
    var type = TreeFactory.GetTreeType("Pino", "Verde", heavyTexture);
    forest.Add(new Tree(i, i * 2, type));
}

forest[0].Draw();
forest[1].Draw();
```

Patrón Behavioral: Strategy

El patrón Strategy (Estrategia) es, posiblemente, el que más impacto tiene en la calidad del código a largo plazo, porque es el "asesino" oficial de los bloques `if/else` y `switch` gigantes.

El Concepto

Permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo utilizan.

El Escenario: Sistema de Descuentos

Imagina que tienes un E-commerce y necesitas aplicar diferentes estrategias de descuento según la temporada:

1. Sin descuento (Normal).
2. Descuento fijo (Ej: -10 USD).
3. Descuento porcentual (Ej: -15%).

Sin Strategy, tendrías un método con un `switch` que crece cada vez que marketing inventa un descuento nuevo.

1. La Interfaz (Strategy)

Define cómo se debe comportar cualquier estrategia de descuento.

```
public interface IDiscountStrategy  
{  
  
    decimal ApplyDiscount (decimal total);  
  
}
```

2. Estrategias Concretas

Cada una implementa su propia lógica.

```
// Estrategia 1: Sin descuento
public class NoDiscount : IDiscountStrategy
{
    public decimal ApplyDiscount(decimal total) => total;
}

// Estrategia 2: Descuento fijo
public class FixedDiscount : IDiscountStrategy
{
    private readonly decimal _amount;
    public FixedDiscount(decimal amount) => _amount = amount;

    public decimal ApplyDiscount(decimal total) => total - _size;
}

// Estrategia 3: Descuento porcentual
public class PercentageDiscount : IDiscountStrategy
{
    private readonly decimal _percentage;
    public PercentageDiscount(decimal percentage) => _percentage = percentage;

    public decimal ApplyDiscount(decimal total) => total - (total * _percentage / 100);
}
```

3. El Contexto

Es la clase que usa la estrategia. No sabe *qué* descuento aplica, solo sabe que tiene un objeto que cumple con `IDiscountStrategy`.

```
public class ShoppingCart
{
    private IDiscountStrategy _strategy;
    public decimal TotalAmount { get; set; }

    // El secreto: pasamos la estrategia por el constructor o un método
    public ShoppingCart(IDiscountStrategy strategy)
    {
        _strategy = strategy;
    }

    // Permite cambiar de estrategia en tiempo de ejecución
    public void SetStrategy(IDiscountStrategy strategy) => _strategy = strategy;

    public decimal CalculateFinalPrice()
    {
        return _strategy.ApplyDiscount(TotalAmount);
    }
}
```

4. Uso en el Código

Fíjate cómo el comportamiento del carrito cambia simplemente cambiando el objeto de estrategia.

```
var cart = new ShoppingCart(new NoDiscount()) { TotalAmount = 100 };  
Console.WriteLine($"Normal: {cart.CalculateFinalPrice()}"); // 100  
  
// Cambiamos a descuento del 50% en tiempo de ejecución  
cart.SetStrategy(new PercentageDiscount(50));  
Console.WriteLine($"Rebajas: {cart.CalculateFinalPrice()}"); // 50  
  
// Cambiamos a cupón de 10 USD  
cart.SetStrategy(new FixedDiscount(10));  
Console.WriteLine($"Cupón: {cart.CalculateFinalPrice()}"); // 90
```

Patrón Behavioral: Observer

El patrón Observer (Observador) es la base de la programación orientada a eventos. Este patrón te interesa porque es la clave para lograr un acoplamiento mínimo: el objeto que genera la noticia no sabe quién la recibe ni qué hacen con ella.

El Concepto

Define una relación uno-a-muchos. Cuando un objeto (llamado Sujeto o Subject) cambia su estado, todos sus dependientes (Observadores) son notificados y actualizados automáticamente.

El Escenario: Sistema de Inventario (Aviso de Stock)

Imagina una tienda online. Un cliente quiere comprar un iPhone pero no hay stock. El cliente pulsa "Avisarme cuando haya stock".

1. El Sujeto es el `Producto`.
2. Los Observadores son los `Usuarios` interesados y quizás un `Sistema de Marketing` que lanza anuncios.

1. La Interfaz del Observador

Define el contrato para todos los que quieran "escuchar".

```
public interface ISubscriber  
{  
  
    void Update (string productName) ;  
  
}
```

2. El Sujeto (El objeto observado)

Mantiene una lista de suscriptores y los notifica.

```
public class Product
{
    private List<ISubscriber> _subscribers = new();
    public string Name { get; }
    public bool IsInStock { get; private set; }

    public Product(string name) => Name = name;

    public void Attach(ISubscriber subscriber) => _subscribers.Add(subscriber);
    public void Detach(ISubscriber subscriber) => _subscribers.Remove(subscriber);

    public void SetStock(bool available)
    {
        IsInStock = available;
        if (available)
        {
            Notify();
        }
    }

    private void Notify()
    {
        foreach (var sub in _subscribers)
        {
            sub.Update(Name);
        }
    }
}
```

3. Observadores Concretos

Diferentes clases que reaccionan de forma distinta al evento.

```
public class Customer : ISubscriber
{
    private string _email;
    public Customer(string email) => _email = email;

    public void Update(string productName) =>
        Console.WriteLine($"Email enviado a {_email}: ¡El {productName} ya está disponible!");
}

public class MarketingDepartment : ISubscriber
{
    public void Update(string productName) =>
        Console.WriteLine($"Marketing: Creando campaña de anuncios para {productName}.");
}
```

4. La forma "Senior" en C#: Events y Delegates

En C#, rara vez verás el patrón Observer implementado con interfaces manuales como arriba, porque el lenguaje tiene soporte nativo mediante `event` y `delegate`.

```
public class ModernProduct
{
    public string Name { get; }
    // Definimos el evento usando EventHandler (estándar de .NET)
    public event EventHandler OnBackInStock;

    public ModernProduct(string name) => Name = name;

    public void SetStock(bool available)
    {
        if (available)
        {
            // Disparamos el evento
            OnBackInStock?.Invoke(this, EventArgs.Empty);
        }
    }
}

// Uso:
var iphone = new ModernProduct("iPhone 15");
iphone.OnBackInStock += (sender, e) => Console.WriteLine(";Notificado vía evento!");
iphone.SetStock(true);
```

Patrón Behavioral: Command

El objetivo del patrón Command es encapsular una petición como un objeto, permitiéndote parametrizar a los clientes con diferentes peticiones, encolar peticiones y soportar operaciones que se pueden deshacer (Undo).

El Escenario: Sistema de Domótica (Smart Home)

Imagina que tienes un control remoto universal. Este control tiene botones, pero no quieres que el control remoto sepa *cómo* encender una luz, *cómo* abrir una persiana o *cómo* encender el aire acondicionado.

Si mañana añades una cafetera, no quieres tener que modificar la clase `RemoteControl`.

1. La Interfaz (Command)

Todos los comandos deben implementar esta interfaz.

csharp

```
public interface ICommand
{
    void Execute();
    void Undo(); // Para soportar el "deshacer"
}
```

2. El Receptor (Receiver)

Es la clase que realmente sabe cómo hacer el trabajo.

csharp



```
public class Light
{
    public void On() => Console.WriteLine("Luz encendida.");
    public void Off() => Console.WriteLine("Luz apagada.");
}
```

3. Comandos Concretos

Encapsulan la acción y el receptor.

csharp



```
public class LightOnCommand : ICommand
{
    private readonly Light _light;
    public LightOnCommand(Light light) => _light = light;

    public void Execute() => _light.On();
    public void Undo() => _light.Off();
}
```


4. El Invocador (Invoker)

Es el que dispara el comando. Fíjate que no sabe qué hace el comando, solo llama a `Execute()`.

csharp

```
public class RemoteControl
{
    private ICommand _lastCommand;

    public void SetCommand(ICommand command) => _lastCommand = command;

    public void PressButton() => _lastCommand?.Execute();

    public void PressUndo() => _lastCommand?.Undo();
}
```

Patrón Behavioral: State (Estado)

El patrón State permite que un objeto altere su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara de clase en tiempo de ejecución.

En lugar de que el objeto principal tenga toda la lógica de todos los estados posibles, delegamos la lógica a objetos "Estado" separados.

El Escenario: Pedido de E-commerce (Order Workflow)

Un pedido puede estar en varios estados: Borrador (Draft), Pagado (Paid) y Enviado (Shipped).

- Si el pedido es *Borrador*, puedes cancelarlo, pero no puedes enviarlo.
- Si el pedido es *Pagado*, no puedes volver a pagarlo, pero puedes enviarlo.
- Si el pedido es *Enviado*, no puedes cancelarlo ni volver a enviarlo.

1. La Interfaz del Estado

Define todas las acciones posibles que se pueden realizar sobre el pedido.

csharp

```
public abstract class OrderState
{
    protected OrderContext _order;
    public void SetContext(OrderContext order) => _order = order;

    public abstract void Cancel();
    public abstract void Pay();
    public abstract void Ship();
}
```

2. El Contexto

Es la clase que el cliente usa. Mantiene una referencia al estado actual.

```
csharp

public class OrderContext
{
    private OrderState _state;

    public OrderContext(OrderState initialState)
    {
        TransitionTo(initialState);
    }

    public void TransitionTo(OrderState state)
    {
        Console.WriteLine($"Contexto: Transición a {state.GetType().Name}.");
        _state = state;
        _state.SetContext(this);
    }

    // El contexto delega el trabajo al estado actual
    public void Cancel() => _state.Cancel();
    public void Pay() => _state.Pay();
    public void Ship() => _state.Ship();
}
```

3. Estados Concretos

Aquí es donde vive la lógica. Cada estado sabe qué está permitido y qué no.

```
csharp

public class DraftState : OrderState
{
    public override void Cancel() => Console.WriteLine("Pedido cancelado con éxito.");
    public override void Pay()
    {
        Console.WriteLine("Procesando pago...");
        _order.TransitionTo(new PaidState()); // Cambiamos de estado
    }
    public override void Ship() => Console.WriteLine("ERROR: No se puede enviar un pedido no pagado.");
}

public class PaidState : OrderState
{
    public override void Cancel() => Console.WriteLine("Pedido cancelado. Reembolsando dinero...");
    public override void Pay() => Console.WriteLine("ERROR: El pedido ya está pagado.");
    public override void Ship()
    {
        Console.WriteLine("Enviando paquete...");
        _order.TransitionTo(new ShippedState());
    }
}

public class ShippedState : OrderState
{
    public override void Cancel() => Console.WriteLine("ERROR: No se puede cancelar, ya está en camino.");
    public override void Pay() => Console.WriteLine("ERROR: Ya fue pagado y enviado.");
    public override void Ship() => Console.WriteLine("ERROR: Ya se envió.");
}
```

4. Uso en el Código

Fíjate que el cliente solo llama a métodos de `OrderContext`, pero el resultado cambia radicalmente según en qué punto del proceso esté.

csharp



```
var order = new OrderContext(new DraftState());

order.Ship(); // ERROR: No se puede enviar...
order.Pay();  // Transición a PaidState
order.Ship(); // Transición a ShippedState
order.Cancel(); // ERROR: No se puede cancelar...
```

Patrón Behavioral: Chain of Responsibility

Permite pasar una solicitud a lo largo de una cadena de "manejadores" (handlers). Al recibir la solicitud, cada manejador decide si la procesa o si la pasa al siguiente manejador de la cadena.

Lo clave aquí es el desacoplamiento: el emisor de la petición no sabe qué objetos la procesarán.

El Escenario: Sistema de Aprobación de Gastos

Imagina que un empleado registra un gasto en la empresa:

1. Si el gasto es $< \$100$, lo aprueba un Líder de Equipo.
2. Si es $< \$1,000$, lo aprueba un Director.
3. Si es $> \$1,000$, lo aprueba el CEO.

1. La Interfaz del Manejador (Handler)

Define el método para procesar y la capacidad de conocer al siguiente en la cadena.

csharp



```
public abstract class Approver
{
    protected Approver _nextHandler;

    public void SetNext(Approver next) => _nextHandler = next;

    public abstract void ProcessRequest(decimal amount);
}
```


2. Manejadores Concretos

Cada uno decide si se hace cargo o "pasa la bola".

```
csharp

public class TeamLeader : Approver
{
    public override void ProcessRequest(decimal amount)
    {
        if (amount < 100)
            Console.WriteLine($"Líder de Equipo: Yo apruebo estos ${amount}.");
        else
            _nextHandler?.ProcessRequest(amount); // Pasa al siguiente
    }
}

public class Director : Approver
{
    public override void ProcessRequest(decimal amount)
    {
        if (amount < 1000)
            Console.WriteLine($"Director: Yo apruebo estos ${amount}.");
        else
            _nextHandler?.ProcessRequest(amount);
    }
}

public class CEO : Approver
{
    public override void ProcessRequest(decimal amount)
    {
        if (amount >= 1000)
            Console.WriteLine($"CEO: Yo apruebo estos ${amount}. Es mucho dinero, pero OK.");
    }
}
```

3. Configuración de la Cadena y Uso

Fíjate que la cadena se construye en el "Composition Root" (donde arranca la app).

csharp

// 1. Creamos los eslabones

var leader = new TeamLeader();

var director = new Director();

var ceo = new CEO();

// 2. Armamos la cadena

leader.SetNext(director);

director.SetNext(ceo);

// 3. Enviamos peticiones al primer eslabón

leader.ProcessRequest(50); // Procesa el Líder

leader.ProcessRequest(500); // Pasa por Líder -> Procesa Director

leader.ProcessRequest(5000); // Pasa por Líder -> Director -> Procesa CEO

Patrón Behavioral: Mediator

El Mediator restringe la comunicación directa entre los objetos y los obliga a colaborar únicamente a través de un objeto mediador.

La analogía perfecta: Una torre de control en un aeropuerto. Los pilotos no hablan entre ellos para decidir quién aterriza; todos hablan con la torre, y la torre decide y coordina. Sin la torre, cada piloto tendría que estar pendiente de otros 50 aviones (acoplamiento insoportable).

El Escenario: Componentes de una Interfaz (o Servicios)

Imagina que tienes un formulario con:

1. Un `Checkbox` de "Soy mayor de edad".
2. Un `Button` de "Comprar alcohol".
3. Un `TextBox` de "Dirección de envío".

Sin Mediator, el `Checkbox` tendría que tener una referencia al `Button` para habilitarlo. El `Button` tendría que saber del `TextBox` para validar. Pronto, todos conocen a todos.

1. La Interfaz del Mediador

csharp



```
public interface IMediator
{
    void Notify(object sender, string @event);
}
```

2. Los Componentes (Colleagues)

Ellos no saben quiénes son sus compañeros, solo conocen al mediador.

```
csharp

public abstract class BaseComponent
{
    protected IMediator _mediator;
    public BaseComponent(IMediator mediator) => _mediator = mediator;
}

public class Checkbox : BaseComponent
{
    public Checkbox(IMediator mediator) : base(mediator) { }

    public void Check()
    {
        Console.WriteLine("Checkbox: Marcado.");
        _mediator.Notify(this, "check");
    }
}

public class Button : BaseComponent
{
    public Button(IMediator mediator) : base(mediator) { }
    public bool Enabled { get; set; }

    public void Click() => Console.WriteLine("Button: Click realizado.");
}
```

3. El Mediador Concreto

Aquí es donde vive la lógica de "quién habla con quién".

```
csharp

public class RegistrationMediator : IMediator
{
    private Checkbox _checkbox;
    private Button _button;

    public RegistrationMediator(Checkbox checkbox, Button button)
    {
        _checkbox = checkbox;
        _button = button;
        _button.Enabled = false; // Estado inicial
    }

    public void Notify(object sender, string @event)
    {
        if (@event == "check")
        {
            Console.WriteLine("Mediador: Reaccionando al check... Habilitando botón.");
            _button.Enabled = true;
        }
    }
}
```

Bonus rápido: Patrón Iterator

Es el patrón que permite recorrer una colección sin exponer su estructura interna (si es una lista, un árbol o un array).

En C#, el Iterator está tan integrado que es "invisible":

- `IEnumerable<T>` es la interfaz del Agregado.
- `IEnumerator<T>` es la interfaz del Iterador.
- `yield return` es la forma mágica de C# de crear iteradores sin escribir toda la clase manual.

Ejemplo: Cuando haces un `foreach`, el compilador de C# lo traduce a llamadas de los métodos `MoveNext()` y `Current` del patrón Iterator.

Patrón Behavioral: Template Method

El Template Method define el esqueleto de un algoritmo en una clase base, pero delega los pasos específicos a las subclases.

Es la implementación pura del "Principio de Hollywood": *"No nos llames, nosotros te llamaremos"*. La clase base controla el flujo (el cuándo) y las subclases proporcionan la implementación (el qué).

El Escenario: Procesador de Documentos

Imagina que tienes que leer diferentes tipos de archivos (PDF, CSV, JSON) y realizar un proceso:

1. Abrir archivo.
2. Extraer datos.
3. Analizar datos.
4. Cerrar archivo.

Los pasos 1, 3 y 4 suelen ser iguales o muy parecidos, pero el paso 2 (extraer) es totalmente diferente para cada formato.

1. La Clase Abstracta (Template)

Aquí definimos el algoritmo principal como un método `public` (y usualmente `sealed` para que nadie cambie el orden de los pasos).

```
csharp

public abstract class DataMiner
{
    // Este es el "Template Method"
    public void Mine(string path)
    {
        OpenFile(path);
        ExtractData();
        AnalyzeData();
        CloseFile();
    }

    // Pasos comunes con implementación por defecto
    protected void OpenFile(string path) => Console.WriteLine($"Abriendo archivo en {path}...");
    protected void CloseFile() => Console.WriteLine("Cerrando archivo.");

    // Paso común
    protected void AnalyzeData() => Console.WriteLine("Analizando estructura de datos...");

    // Paso que CADA subclase debe implementar (el "hueco" en la plantilla)
    protected abstract void ExtractData();

    // "Hook" (Gancho): Un método opcional que las subclases pueden o no sobrescribir
    protected virtual void OnProcessFinished() { }
}
```

2. Implementaciones Concretas

Solo se preocupan de su parte específica.

csharp



```
public class PdfDataMiner : DataMiner
{
    protected override void ExtractData()
    {
        Console.WriteLine("Extrayendo texto del PDF usando motor de OCR...");
    }
}

public class CsvDataMiner : DataMiner
{
    protected override void ExtractData()
    {
        Console.WriteLine("Parseando filas y columnas del CSV...");
    }
}
```

3. Uso en el Código

El cliente no llama a los pasos individuales, llama al método plantilla.

csharp

```
DataMiner miner = new PdfDataMiner();  
miner.Mine("factura.pdf");  
  
Console.WriteLine("---");  
  
miner = new CsvDataMiner();  
miner.Mine("ventas.csv");
```



Patrón Behavioral: Visitor

El Visitor permite separar un algoritmo de la estructura de objetos sobre la que opera.

La clave: Imagina que tienes una estructura de objetos ya terminada (como un árbol de clases). De repente, necesitas añadir una función de "Exportar a XML", luego una de "Calcular Impuestos" y luego una de "Auditoría". Si metes todos esos métodos en las clases originales, las vas a "ensuciar" con lógica que no les pertenece. El Visitor te permite poner esa lógica en clases externas.

El Escenario: Sistema de Seguros

Tienes diferentes tipos de clientes: Personas Físicas y Grandes Empresas. Quieres aplicarles diferentes algoritmos: uno para "Enviar Oferta Comercial" y otro para "Calcular Riesgo".

1. La Interfaz del Elemento (El que recibe la visita)

Aquí es donde ocurre el truco del Double Dispatch.

csharp

```
public interface IClient
{
    // Este método es La "puerta" para el visitante
    void Accept(IVisitor visitor);
}
```

2. Elementos Concretos

csharp



```
public class Person : IClient
{
    public string Name { get; set; }
    public void Accept(IVisitor visitor) => visitor.VisitPerson(this);
}

public class Company : IClient
{
    public string BusinessName { get; set; }
    public int EmployeeCount { get; set; }
    public void Accept(IVisitor visitor) => visitor.VisitCompany(this);
}
```

4. Visitantes Concretos (Los Algoritmos)

Aquí es donde vive la lógica que normalmente "ensuciaría" las clases de arriba.

csharp



// Algoritmo 1: Enviar Ofertas

```
public class SalesVisitor : IVisitor
```

```
{
```

```
    public void VisitPerson(Person person) =>
```

```
        Console.WriteLine($"Enviando cupón de descuento minorista a {person.Name}.");
```

```
    public void VisitCompany(Company company) =>
```

```
        Console.WriteLine($"Enviando propuesta de suscripción corporativa a {company.BusinessName}.");
```

```
}
```

// Algoritmo 2: Calcular Riesgo

```
public class RiskAnalysisVisitor : IVisitor
```

```
{
```

```
    public void VisitPerson(Person person) =>
```

```
        Console.WriteLine($"Analizando historial crediticio de {person.Name}.");
```

```
    public void VisitCompany(Company company) =>
```

```
        Console.WriteLine($"Analizando balances de {company.BusinessName} con {company.EmployeeCount} emplea
```

```
}
```

5. Uso en el Código

Fíjate cómo podemos pasar diferentes "visitantes" a la misma lista de objetos.

```
csharp

var clients = new List<IClient> {
    new Person { Name = "Juan" },
    new Company { BusinessName = "Tech Corp", EmployeeCount = 500 }
};

var sales = new SalesVisitor();
var risk = new RiskAnalysisVisitor();

// Aplicamos ventas a todos
foreach (var client in clients) {
    client.Accept(sales);
}

// Aplicamos riesgo a todos (sin haber cambiado las clases Person/Company)
foreach (var client in clients) {
    client.Accept(risk);
}
```


Anti Patrones

1. Antipatrones de Desarrollo (Código)

The God Object (El Objeto Todopoderoso)

- **Qué es:** Una clase que hace demasiadas cosas, tiene miles de líneas de código y conoce demasiados detalles del sistema. Viola el Principio de Responsabilidad Única (SRP).
- **Ejemplo en C#:** Una clase `OrderManager` que valida el stock, procesa el pago, envía el email, guarda en la base de datos y genera el PDF de la factura.
- **Consecuencia:** Es imposible de testear, cualquier cambio rompe algo inesperado y nadie se atreve a tocarla.

Golden Hammer (El Martillo de Oro)

- **Qué es:** Estar tan obsesionado con una tecnología o patrón que intentas aplicarlo a todo, aunque no encaje.
- **Ejemplo:** "Como acabamos de aprender Microservicios, vamos a dividir esta aplicación de 3 usuarios en 15 servicios separados" o "Vamos a usar el patrón Visitor para sumar dos números".
- **Consecuencia:** Sobreingeniería (Overengineering) y complejidad innecesaria.

Spaghetti Code vs. Lasagna Code

- **Spaghetti:** Código sin estructura, con saltos lógicos y dependencias enredadas (clásico de principiantes).
- **Lasagna:** El error del Senior. Demasiadas capas de abstracción. Para llegar a la lógica de base de datos tienes que pasar por la Interfaz, el Proxy, el Decorator, el Bridge, el Service y el Repository.
- **Consecuencia:** En el Spaghetti no sabes dónde estás; en el Lasagna te pierdes entre capas para hacer un simple "Hello World".

2. Antipatrones Arquitectónicos

Big Ball of Mud (Gran Bola de Lodo)

- **Qué es:** Un sistema que carece de una arquitectura definible. Los datos fluyen por todos lados, no hay límites entre módulos y el diseño es puramente reactivo ("parche sobre parche").
- **Ejemplo:** Una aplicación donde la base de datos tiene lógica de negocio (Triggers), el Frontend tiene lógica de negocio (JS) y el Backend también, pero ninguno está coordinado.

Stovepipe System (Sistema de Cañerías)

- **Qué es:** Crear módulos o servicios que están tan aislados y son tan específicos que no pueden reutilizarse ni comunicarse con otros de forma estándar. Suele pasar cuando diferentes equipos trabajan sin hablar entre ellos.

Vendor Lock-in (Dependencia del Proveedor)

- **Qué es:** Diseñar tu sistema de forma que dependa totalmente de las características propietarias de un proveedor (AWS, Azure, Oracle).
- **Ejemplo:** Usar funciones específicas de SQL Server dentro de tu código en lugar de usar una capa de abstracción, lo que hace imposible migrar a PostgreSQL en el futuro.

3. Antipatrones de Diseño (Específicos)

Soft Coding

- **Qué es:** El miedo a "hardcodear" llevado al extremo. Intentas que absolutamente todo sea configurable (en el `appsettings.json` o base de datos), incluyendo la lógica de negocio.
- **Ejemplo:** Un motor de reglas donde el usuario puede escribir scripts complejos en la base de datos para decidir el precio.
- **Consecuencia:** El código se vuelve ilegible, difícil de debuguear y terminas escribiendo un lenguaje de programación malo dentro de tu configuración.

Boat Anchor (Ancla de Barco)

- **Qué es:** Mantener código, librerías o servidores "por si acaso" los necesitamos en el futuro, aunque no se usen hoy.
- **Consecuencia:** Ruido visual, binarios más pesados y confusión para los nuevos desarrolladores que intentan entender qué partes del código están vivas.

Ghost Objects (Objetos Fantasma / Poltergeists)

- **Qué es:** Clases que no tienen una responsabilidad real, solo sirven para pasar información de una clase A a una clase B. Suelen ser efímeras y solo ensucian el diagrama de clases.

4. Antipatrones de Gestión (Contexto Senior)

Death March (Marcha de la Muerte)

- **Qué es:** Un proyecto donde todos saben que el alcance es imposible y los recursos insuficientes, pero se obliga al equipo a trabajar horas extras bajo una presión insostenible para intentar llegar a una fecha arbitraria.

Smoke and Mirrors (Humo y Espejos)

- **Qué es:** Mostrar funcionalidades que no existen en una demo para impresionar a los stakeholders, sabiendo que por detrás el código es un desastre o ni siquiera funciona.